HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI THỰC HÀNH 1

HỌC PHẦN: IOT VÀ ỨNG DỤNG

Sinh viên thực hiện:

B23DCAT211 Nguyễn Trung Nghĩa

Tên lớp: Nhóm 13

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

HÀ NỘI 2026

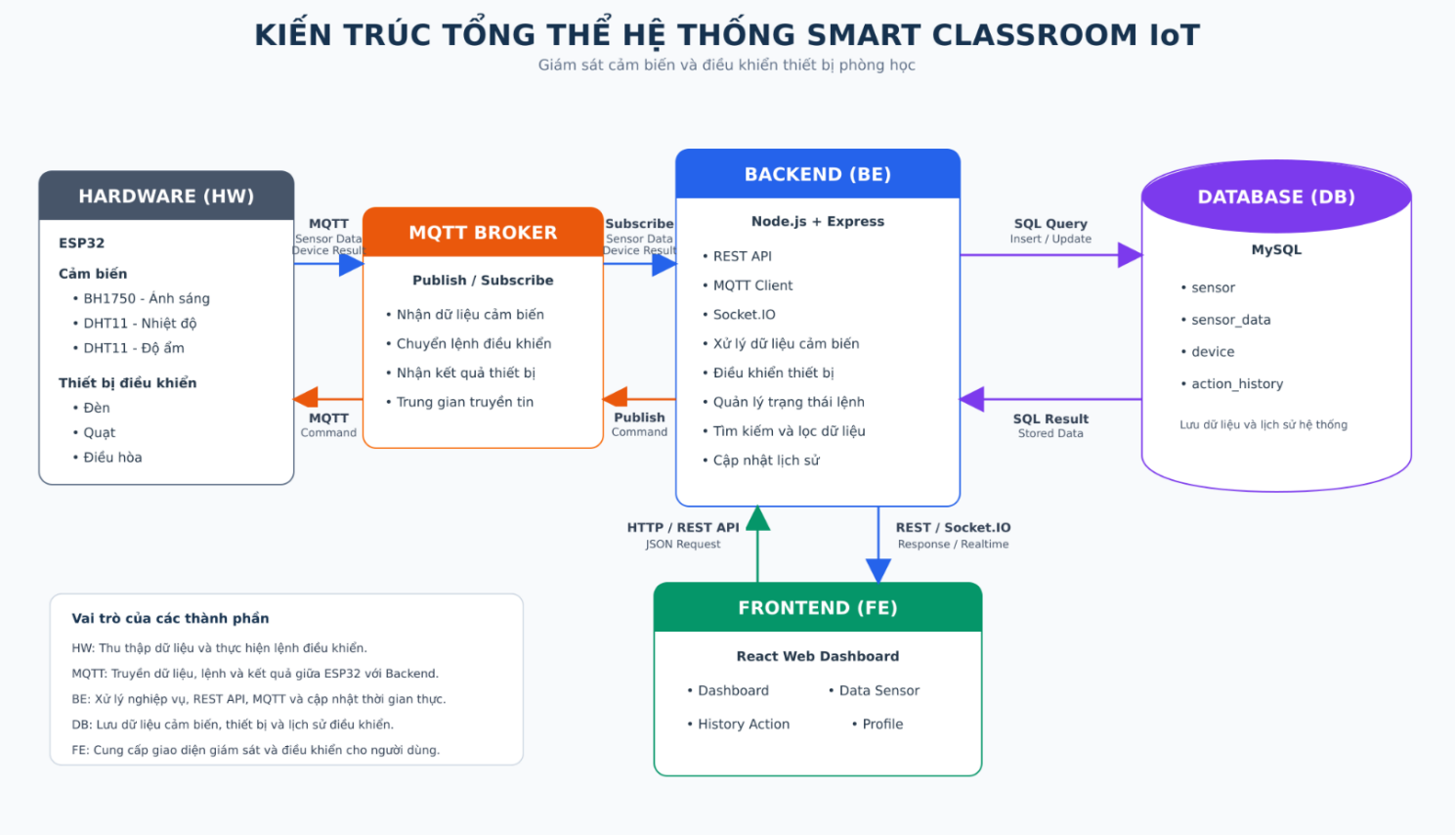
MỤC LỤC

MỤC LỤC 2
CHƯƠNG 1. Kiến trúc tổng quan hệ thống 3
1.1 Usecase tổng quan 4
1.2 UC01 – Xem dữ liệu cảm biến thời gian thực 5
1.3 UC02 – Xem biểu đồ dữ liệu cảm biến 5
1.4 UC03 – Điều khiển thiết bị 6
1.5 UC04 – Tra cứu dữ liệu cảm biến 7
1.6 UC05 – Tra cứu lịch sử điều khiển 8
1.7 UC06 – Xem hồ sơ và tài liệu dự án 9
CHƯƠNG 2. Thiết kế giao diện 10
2.1 Giao diện Dashboard 10
2.2 Giao diện lịch sử cảm biến 10
2.3 Giao diện lịch sử điều khiển 11
2.4 Giao diện profile cá nhân 11
CHƯƠNG 3. Thiết kế Database 12
CHƯƠNG 4. Thiết kế API 13
4.1 API 1: Lấy cảm biến mới nhất 14
4.2 API 2: Lấy dữ liệu biểu đồ 15
4.3 API 3: Lấy lịch sử cảm biến 16
4.4 API 4: Lấy trạng thái thiết bị 17
4.5 API 5: Điều khiển thiết bị 18
4.6 API 6: Lấy lịch sử điều khiển 20
CHƯƠNG 5. Sequence Diagram 21
5.1 UC03 - Điều khiển thiết bị 21
5.2 UC01 – Xem dữ liệu cảm biến thời gian thực 22
5.3 UC02 – Xem biểu đồ dữ liệu cảm biến 22
5.4 UC04 -Tra cứu dữ liệu cảm biến 23
5.5 UC05 – Tra cứu lịch sử điều khiển 24

CHƯƠNG 1. KIẾN TRÚC TỔNG QUAN HỆ THỐNG

Hệ thống Smart Classroom IoT được xây dựng theo kiến trúc nhiều lớp, gồm thiết bị ESP32, MQTT Broker, Backend, Frontend và Database. ESP32 có nhiệm vụ thu thập dữ liệu từ các cảm biến ánh sáng, nhiệt độ và độ ẩm, sau đó gửi dữ liệu đến MQTT Broker. Backend đăng ký nhận dữ liệu từ MQTT Broker, xử lý và lưu dữ liệu vào Database. Frontend giao tiếp với Backend thông qua REST API để lấy dữ liệu ban đầu và sử dụng Socket.IO để nhận dữ liệu cập nhật theo thời gian thực.

Khi người dùng điều khiển đèn, quạt hoặc điều hòa, Frontend gửi yêu cầu đến Backend. Backend tạo một bản ghi lịch sử với trạng thái PENDING rồi gửi lệnh điều khiển qua MQTT Broker. ESP32 nhận lệnh, thực hiện bật hoặc tắt thiết bị và gửi kết quả phản hồi. Backend cập nhật trạng thái thành SUCCESS hoặc FAILED trong Database, sau đó thông báo kết quả cho Frontend thông qua Socket.IO. Kiến trúc này giúp tách biệt giao diện, xử lý nghiệp vụ, truyền thông IoT và lưu trữ dữ liệu, từ đó thuận tiện cho việc phát triển và bảo trì hệ thống.

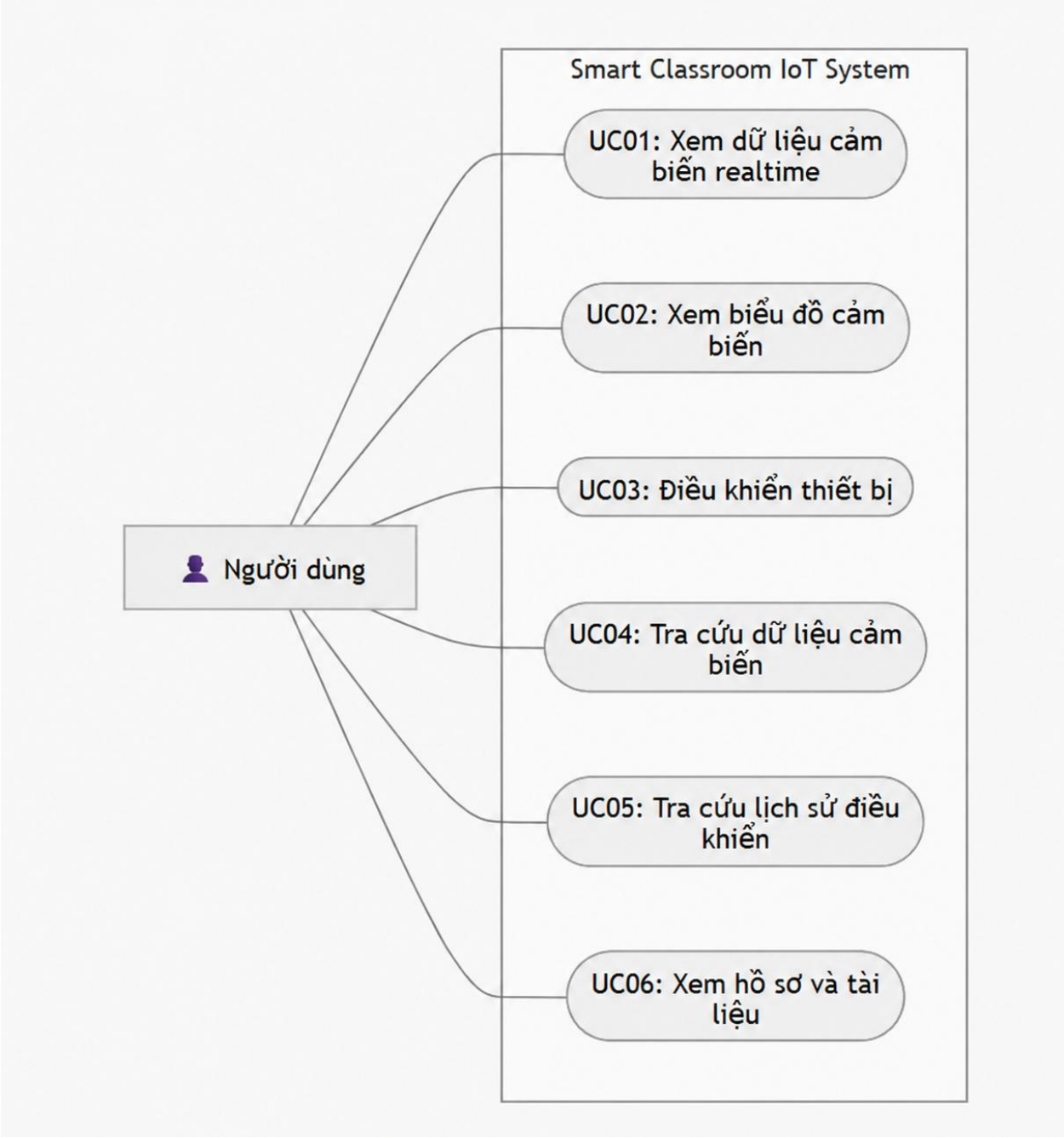


Thành phần và công nghệ lựa chọn

Nhóm	Thành phần
Phần cứng	ESP32, DHT11, cảm biến ánh sáng, LED, điện trở và dây nối
Frontend	HTML, CSS, JavaScript, Chart.js

Nhóm	Thành phần
Backend	Node.js, Express, MQTT.js
Giao tiếp web	REST API và WebSocket
Giao tiếp thiết bị	MQTT
MQTT Broker	Eclipse Mosquitto
Cơ sở dữ liệu	MySQL

1.1 Usecase tổng quan



1.2 UC01 – Xem dữ liệu cảm biến thời gian thực

Tên Use Case	Xem dữ liệu cảm biến thời gian thực
Actor	Người dùng
Mô tả	Người dùng xem giá trị ánh sáng, nhiệt độ và độ ẩm mới nhất trên Dashboard
Tiền điều kiện	Người dùng truy cập được hệ thống
Sự kiện kích hoạt	Người dùng mở màn hình Dashboard
Hậu điều kiện	Dữ liệu cảm biến mới nhất được hiển thị

Kịch bản chính:

1. Người dùng chọn **Dashboard**.
2. Hệ thống yêu cầu dữ liệu cảm biến mới nhất.
3. Hệ thống nhận dữ liệu ánh sáng, nhiệt độ và độ ẩm.
4. Hệ thống hiển thị giá trị tại ba thẻ cảm biến.

Ngoại lệ:

- 4.1: Nếu mất kết nối, hệ thống thông báo **"Không thể kết nối đến máy chủ"**.

1.3 UC02 – Xem biểu đồ dữ liệu cảm biến

Tên Use Case	Xem biểu đồ dữ liệu cảm biến
Actor	Người dùng
Mô tả	Người dùng theo dõi sự thay đổi của ánh sáng, nhiệt độ và độ ẩm theo thời gian thông qua biểu đồ đường
Tiền điều kiện	Người dùng truy cập được hệ thống; hệ thống đã có dữ liệu cảm biến
Sự kiện kích hoạt	Người dùng mở màn hình Dashboard
Hậu điều kiện	Biểu đồ hiển thị dữ liệu của ba loại cảm biến theo thời gian

Kịch bản chính:

1. Người dùng chọn **Dashboard**.
2. Hệ thống yêu cầu dữ liệu cảm biến theo thời gian.
3. Hệ thống nhận dữ liệu ánh sáng, nhiệt độ và độ ẩm.
4. Hệ thống xử lý dữ liệu theo thứ tự thời gian.
5. Hệ thống hiển thị biểu đồ gồm ba đường:

- Ánh sáng.
- Nhiệt độ.
- Độ ẩm.

6. Khi có dữ liệu mới, hệ thống cập nhật biểu đồ.

Luồng ngoại lệ:

- 5.1: Nếu một loại cảm biến chưa có dữ liệu, hệ thống vẫn hiển thị các đường dữ liệu còn lại.
- 5.2: Nếu không có dữ liệu, hệ thống hiển thị thông báo **"Chưa có dữ liệu để hiển thị"**.
- 5.3: Nếu không thể lấy dữ liệu từ máy chủ, hệ thống hiển thị thông báo **"Không thể tải biểu đồ"**.

1.4 UC03 – Điều khiển thiết bị

Tên Use Case	Điều khiển thiết bị
Actor	Người dùng
Mô tả	Người dùng bật hoặc tắt đèn, quạt và điều hòa từ màn hình Dashboard
Tiền điều kiện	Người dùng đang ở màn hình Dashboard; Backend và ESP32 đang hoạt động
Sự kiện kích hoạt	Người dùng nhấn công tắc ON/OFF của một thiết bị
Hậu điều kiện	Trạng thái thiết bị được cập nhật và hành động được lưu vào lịch sử điều khiển

Kịch bản chính:

1. Người dùng chọn một trong ba thiết bị:
   - Đèn.
   - Quạt.
   - Điều hòa.
2. Người dùng nhấn công tắc để bật hoặc tắt thiết bị.
3. Frontend gửi yêu cầu
4. Backend tạo action_history = PENDING
5. Frontend hiển thị "Đang xử lý"

6. Backend gửi lệnh điều khiển đến ESP32.
7. ESP32 thực hiện bật hoặc tắt thiết bị.
8. ESP32 gửi kết quả phản hồi về Backend.
9. Backend cập nhật trạng thái hành động thành **"Thành công"**.
10. Frontend cập nhật trạng thái ON/OFF của thiết bị.
11. Hệ thống hiển thị kết quả cho người dùng.

Luồng ngoại lệ:

- 8.1: Nếu ESP32 phản hồi thực hiện thất bại hoặc không phản hồi trong thời gian quy định, hệ thống hiển thị **"Điều khiển thiết bị thất bại"**.

1.5 UC04 – Tra cứu dữ liệu cảm biến

Tên Use Case	Tra cứu dữ liệu cảm biến
Actor	Người dùng
Mô tả	Người dùng xem, tìm kiếm và sắp xếp các bản ghi dữ liệu cảm biến đã được lưu
Tiền điều kiện	Người dùng truy cập được hệ thống
Sự kiện kích hoạt	Người dùng chọn **Data Sensor** trên thanh menu
Hậu điều kiện	Danh sách dữ liệu phù hợp với yêu cầu tra cứu được hiển thị

Kịch bản chính:

1. Người dùng chọn **Data Sensor**.
2. Hệ thống lấy danh sách dữ liệu cảm biến.
3. Hệ thống hiển thị toàn bộ dữ liệu theo bảng gồm các cột:
    - ID.
    - Tên cảm biến.
    - Loại cảm biến.
    - Giá trị.
    - Thời gian.
4. Người dùng nhập nội dung vào ô tìm kiếm.
5. Người dùng có thể chọn phương thức sắp xếp:
    - Loại cảm biến.

- Giá trị tăng dần.
- Giá trị giảm dần.
- Thời gian mới nhất.
- Thời gian cũ nhất.

6. Người dùng nhấn **Tìm kiếm**.
7. Hệ thống hiển thị danh sách kết quả.
8. Người dùng sử dụng phân trang để xem các bản ghi tiếp theo.

Ngoại lệ:

- 3.1: Nếu không thể lấy dữ liệu từ máy chủ, hệ thống hiển thị **"Không thể tải dữ liệu cảm biến"**.

1.6 UC05 – Tra cứu lịch sử điều khiển

Tên Use Case	Tra cứu lịch sử điều khiển
Actor	Người dùng
Mô tả	Người dùng xem và tìm kiếm lịch sử các lệnh bật hoặc tắt thiết bị
Tiền điều kiện	Người dùng truy cập được hệ thống
Sự kiện kích hoạt	Người dùng chọn **History Action** trên thanh menu
Hậu điều kiện	Danh sách hành động điều khiển phù hợp được hiển thị

Kịch bản chính:

1. Người dùng chọn **History Action**.
2. Hệ thống lấy danh sách lịch sử điều khiển.
3. Hệ thống hiển thị toàn bộ dữ liệu dạng bảng gồm các cột:
   - ID.
   - Tên thiết bị.
   - Hành động.
   - Trạng thái.
   - Thời gian gửi.
   - Thời gian phản hồi.
4. Người dùng nhập ID hoặc tên thiết bị vào ô tìm kiếm.

5. Người dùng nhấn **Tìm kiếm**.
6. Hệ thống tìm các bản ghi phù hợp và hiển thị kết quả tìm kiếm.
7. Người dùng sử dụng phân trang để xem thêm bản ghi.

Luồng ngoại lệ:

- 3.1: Nếu không thể lấy dữ liệu từ máy chủ, hệ thống hiển thị **"Không thể tải lịch sử điều khiển"**.

1.7 UC06 – Xem hồ sơ và tài liệu dự án

Tên Use Case	Xem hồ sơ và tài liệu dự án
Actor	Người dùng
Mô tả	Người dùng xem thông tin sinh viên và truy cập các tài liệu liên quan đến dự án
Tiền điều kiện	Người dùng truy cập được hệ thống
Sự kiện kích hoạt	Người dùng chọn **Profile** trên thanh menu
Hậu điều kiện	Thông tin hồ sơ được hiển thị hoặc tài liệu được mở theo lựa chọn của người dùng

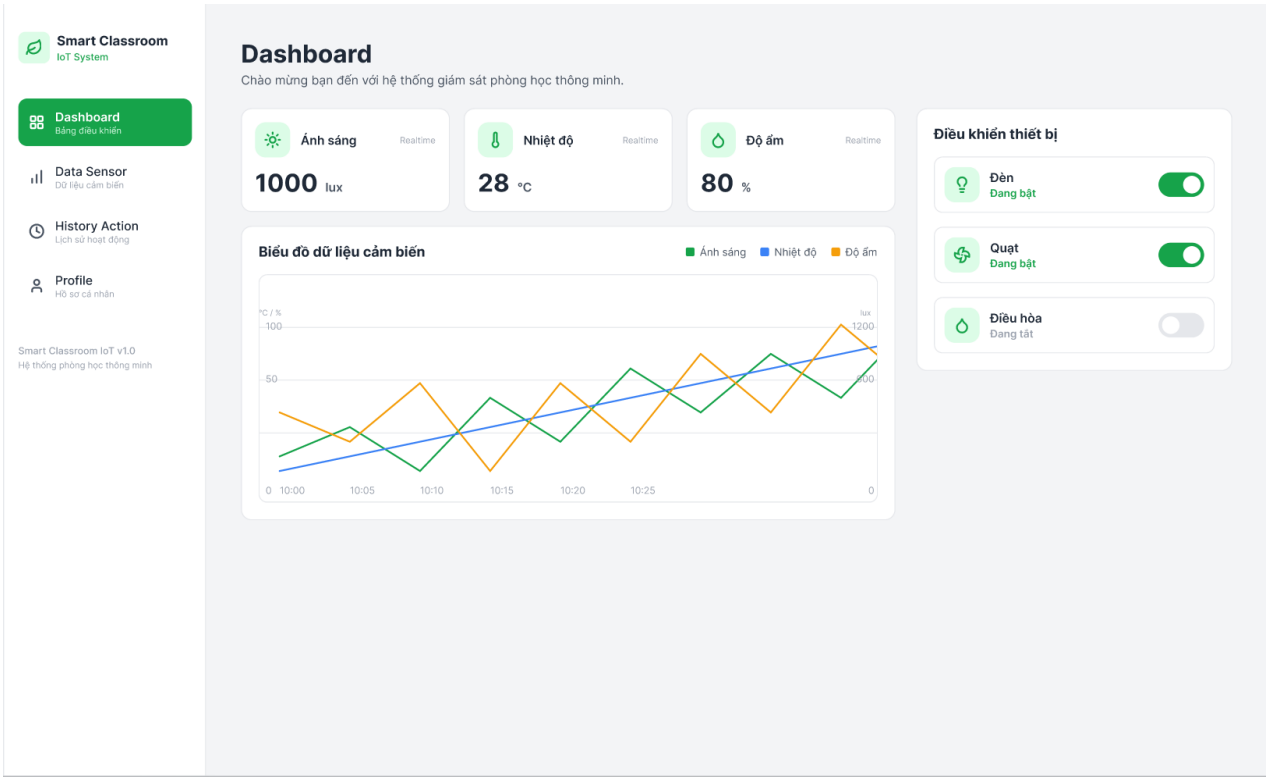
Kịch bản chính:

1. Người dùng chọn **Profile**.
2. Hệ thống hiển thị thông tin cá nhân:
    - Ảnh đại diện.
    - Họ và tên.
    - Mã sinh viên.
3. Hệ thống hiển thị bốn liên kết tài liệu:
    - Figma.
    - Báo cáo PDF.
    - GitHub.
    - Postman/Swagger.
4. Người dùng chọn một tài liệu.
5. Hệ thống mở liên kết hoặc tải tài liệu tương ứng.

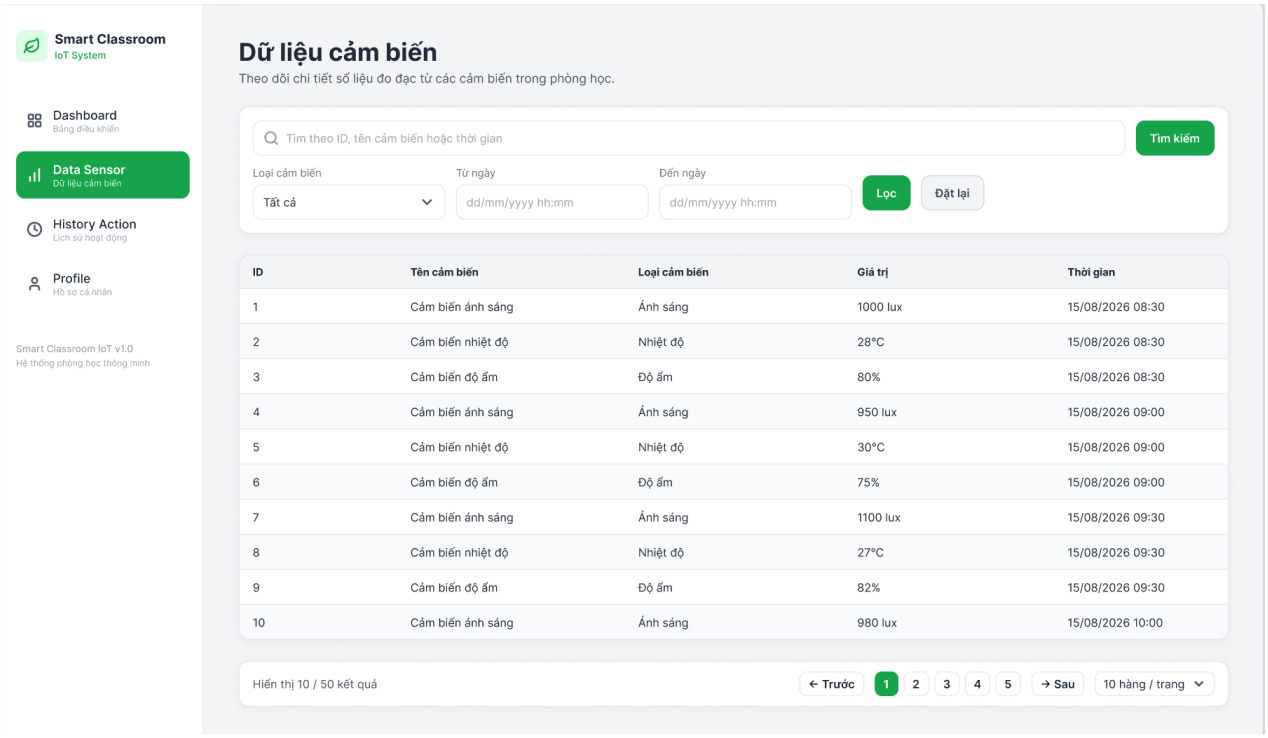
CHƯƠNG 2. THIẾT KẾ GIAO DIỆN

Link figma:
https://www.figma.com/design/Q142HeQDCmSJKCGBhen3nH/Untitled?node-id=0-1&t=OcuxswS7puFmUJfd-1

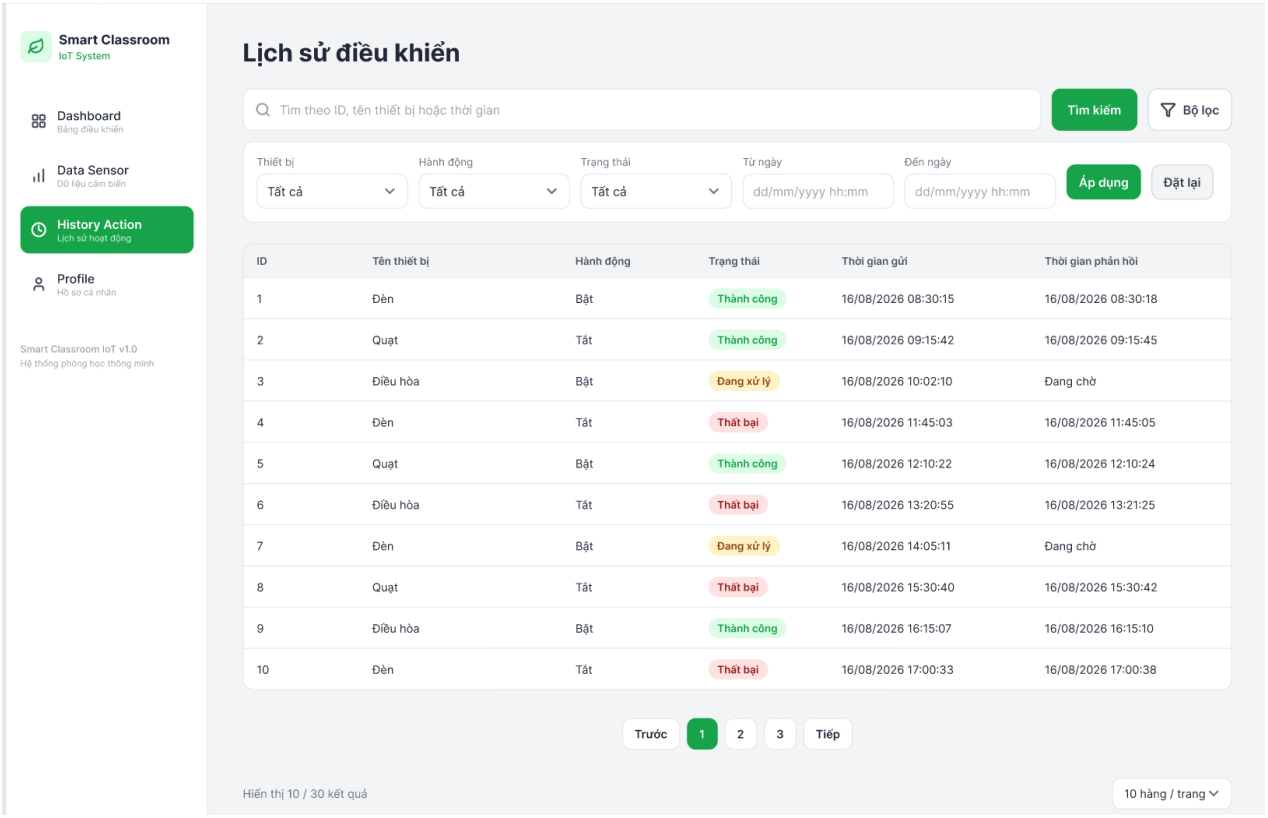
2.1 Giao diện Dashboard



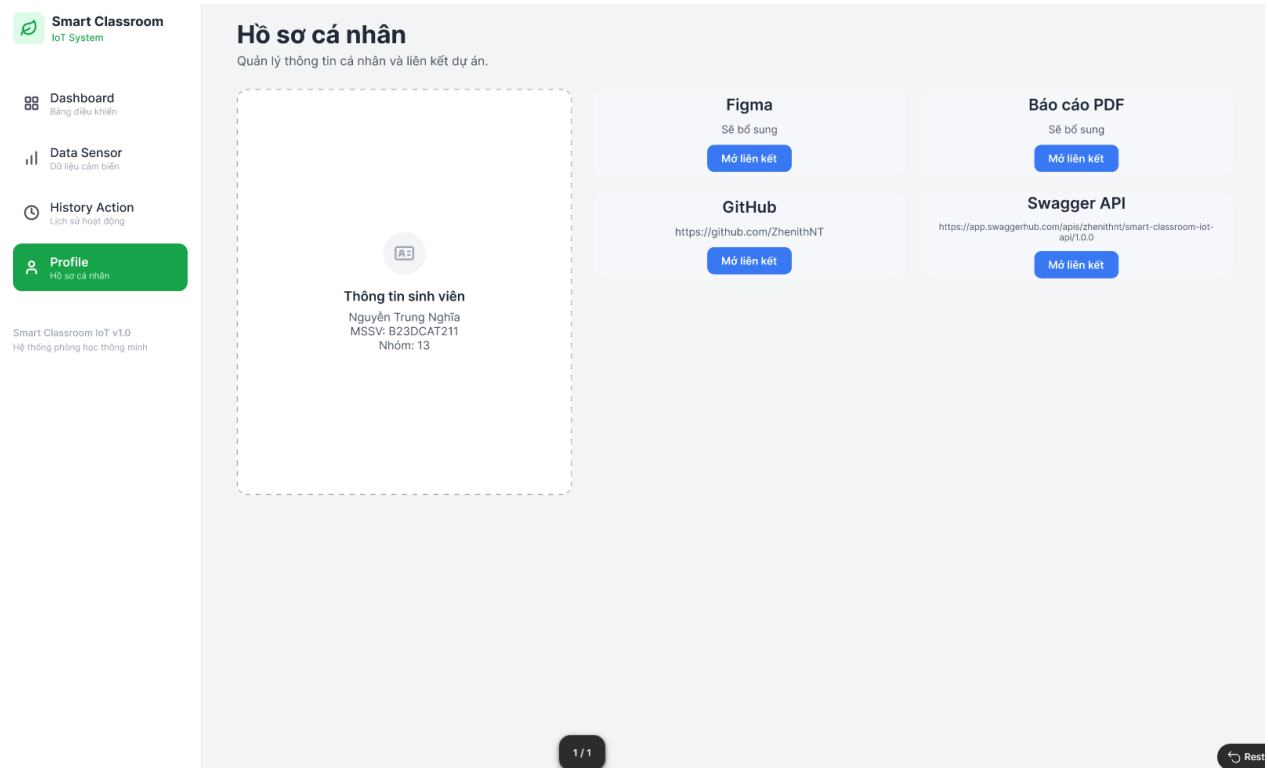
2.2 Giao diện lịch sử cảm biến



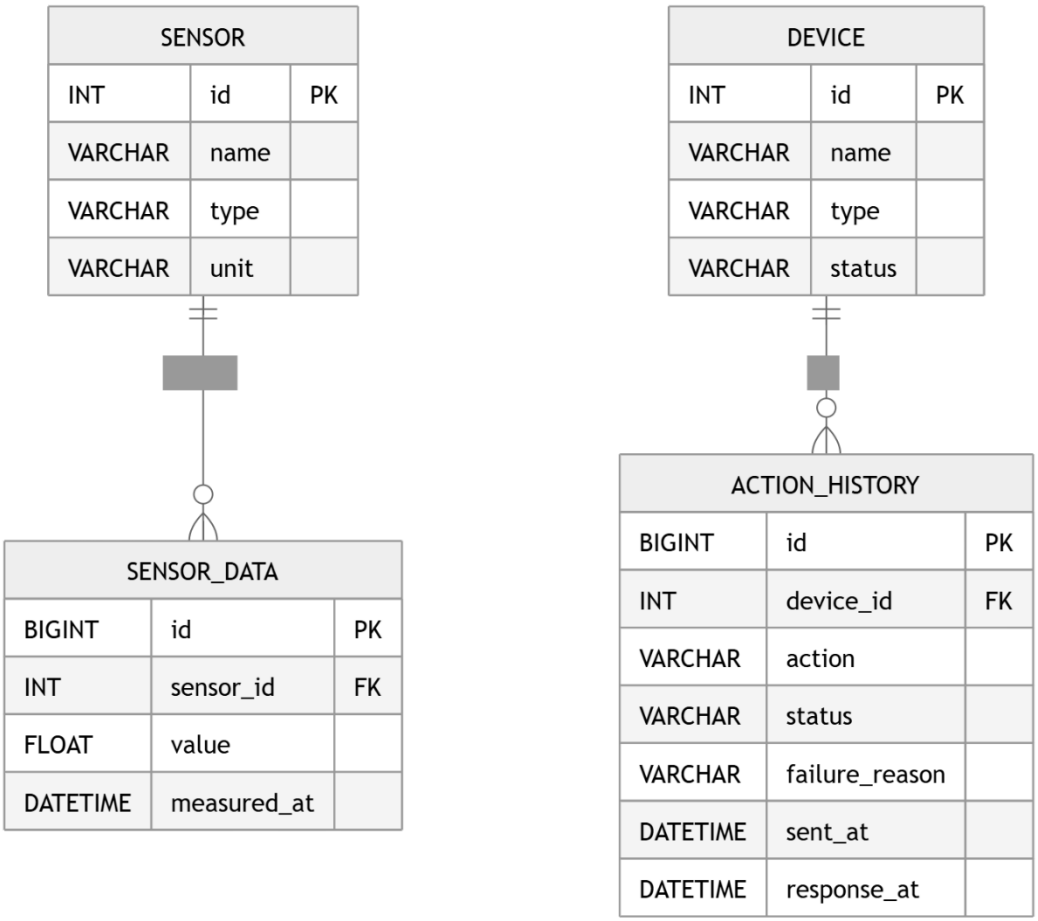
2.3 Giao diện lịch sử điều khiển



2.4 Giao diện profile cá nhân



CHƯƠNG 3. THIẾT KẾ DATABASE



Bảng sensor

Trường	**Kiểu**	**Ý nghĩa**
id	INT, PK	Mã cảm biến
name	VARCHAR	Tên cảm biến
type	VARCHAR	TEMPERATURE, HUMIDITY, LIGHT
unit	VARCHAR	°C, %, lux

Bảng sensor_data

Trường	**Kiểu**	**Ý nghĩa**
id	BIGINT, PK	Mã bản ghi
sensor_id	INT, FK	Cảm biến gửi dữ liệu
value	FLOAT	Giá trị đo
measured_at	DATETIME	Thời gian đo

Bảng device

Trường	Kiểu	Ý nghĩa
id	INT, PK	Mã thiết bị
name	VARCHAR	Đèn, quạt, điều hòa
type	VARCHAR	LIGHT, FAN, AIR_CONDITIONER
status	VARCHAR	ON, OFF

Bảng action_history

Trường	Kiểu	Ý nghĩa
id	BIGINT, PK	Mã lệnh
device_id	INT, FK	Thiết bị nhận lệnh
action	VARCHAR	TURN_ON, TURN_OFF
status	VARCHAR	PENDING, SUCCESS, FAILED
failure_reason	VARCHAR, NULL	Nguyên nhân thất bại
sent_at	DATETIME	Thời gian gửi
response_at	DATETIME, NULL	Thời gian phản hồi

CHƯƠNG 4. THIẾT KẾ API

Link Swagger:

https://app.swaggerhub.com/apis/zhenithnt/smart-classroom-iot-api/1.0.0

Danh sách API hệ thống:

STT	Method	Endpoint	Mục đích
1	GET	/api/v1/sensor-data/latest	Lấy 3 giá trị cảm biến mới nhất
2	GET	/api/v1/sensor-data/chart	Lấy dữ liệu vẽ biểu đồ
3	GET	/api/v1/sensor-data	Lấy lịch sử cảm biến
4	GET	/api/v1/devices	Lấy trạng thái hiện tại của 3 thiết bị
5	POST	/api/v1/devices/{id}/actions	Thay đổi trạng thái thiết bị
6	GET	/api/v1/actions	Lấy lịch sử điều khiển

Các HTTP status cần dùng:

Status	Ý nghĩa
200	Lấy dữ liệu thành công
202	Đã nhận lệnh, đang chờ ESP32
400	Dữ liệu đầu vào không hợp lệ
404	Không tìm thấy cảm biến hoặc thiết bị
409	Thiết bị đang có lệnh chờ xử lý
500	Lỗi Backend
503	MQTT hoặc phần cứng mất kết nối

4.1 API 1: Lấy cảm biến mới nhất

```
GET /api/v1/sensor-data/latest
```

- Hiển thị ba thẻ Ánh sáng, Nhiệt độ và Độ ẩm trên Dashboard.

- Đầu vào: Không

- Đầu ra:

+ Thành công

```
HTTP  200 OK
```

```
{
 "success": true,
 "data": [
   {
     "sensorId": 1,
     "name": "Cảm biến ánh sáng",
     "type": "LIGHT",
     "value": 1000,
     "unit": "lux",
     "measuredAt": "2026-08-18T14:20:00"
   },
   {
     "sensorId": 2,
     "name": "Cảm biến nhiệt độ",
     "type": "TEMPERATURE",
     "value": 28,
     "unit": "°C",
     "measuredAt": "2026-08-18T14:20:00"
   },
   {
     "sensorId": 3,
     "name": "Cảm biến độ ẩm",
     "type": "HUMIDITY",
     "value": 80,
```

```
      "unit": "%",
      "measuredAt": "2026-08-18T14:20:00"
    }
  ]
}
```

+ Thất bại:

```
HTTP 500 Internal Server Error
{
  "success": false,
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "message": "Không thể lấy dữ liệu cảm biến"
  }
}
```

4.2 API 2: Lấy dữ liệu biểu đồ

```
GET /api/v1/sensor-data/chart
```

- Mặc định lấy 20 nhóm dữ liệu gần nhất

- Đầu vào: Không có

- Đầu ra:

+ Thành công:

```
HTTP 200 OK
{
 "success": true,
 "data": [
   {
     "time": "2026-08-18T14:00:00",
     "light": 900,
     "temperature": 27,
     "humidity": 78
   },
   {
     "time": "2026-08-18T14:05:00",
     "light": 1000,
     "temperature": 28,
     "humidity": 80
   }
 ]
}
```

+ Thất bại:

```
HTTP 500 Internal Server Error
```

```
{
  "success": false,
  "error": {
    "code": "CHART_DATA_ERROR",
    "message": "Không thể lấy dữ liệu biểu đồ"
  }
}
```

4.3 API 3: Lấy lịch sử cảm biến

`GET /api/v1/sensor-data`

- Lấy lịch sử cảm biến và tra cứu dữ liệu cảm biến

- Đầu vào:

Tham số	Bắt buộc	Mặc định	Ý nghĩa
page	Không	1	Trang hiện tại
limit	Không	10	Số bản ghi mỗi trang
search	Không	Rỗng	Tìm theo tên cảm biến
type	Không	Tất cả	LIGHT, TEMPERATURE, HUMIDITY
sortBy	Không	measuredAt	type, value, measuredAt
sortOrder	Không	DESC	ASC, DESC

Ví dụ:

```
GET /api/v1/sensor-data?page=1&limit=10&type=TEMPERATURE&sortBy=value&sortOrder=DESC
```

- Đầu ra thành công:

```
HTTP 200 OK
{
 "success": true,
 "data": [
   {
     "id": 101,
      "sensorID": "2"
     "sensorName": "Cảm biến nhiệt độ",
     "sensorType": "TEMPERATURE",
     "value": 28,
     "unit": "°C",
```

```
      "measuredAt": "2026-08-18T14:20:00"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "totalItems": 52,
    "totalPages": 6
  }
}
```

- Sai tham số:

```
HTTP 400 Bad Request
{
  "success": false,
  "error": {
    "code": "INVALID_QUERY_PARAMETER",
    "message": "Tham số sắp xếp không hợp lệ"
  }
}
```

- Lỗi Backend:

```
HTTP 500 Internal Server Error
```

4.4 API 4: Lấy trạng thái thiết bị

```
GET /api/v1/devices
```

- Lấy trạng thái hiện tại của thiết bị

- Đầu vào: Không có

- Đầu ra thành công

```
HTTP 200 OK
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Đèn",
      "type": "LIGHT",
      "status": "ON"
    },
    {
      "id": 2,
      "name": "Quạt",
      "type": "FAN",
      "status": "OFF"
```

```
    },
    {
      "id": 3,
      "name": "Điều hòa",
      "type": "AIR_CONDITIONER",
      "status": "OFF"
    }
  ]
}
```

- Đầu ra thất bại:

```
HTTP 500 Internal Server Error
```

4.5 API 5: Điều khiển thiết bị

```
POST /api/v1/devices/{id}/actions
```

- Điều khiển trạng thái của 3 thiết bị
- Đầu vào thứ nhất: ID trên URL

/devices/1/actions → Đèn
/devices/2/actions → Quạt
/devices/3/actions → Điều hòa

- Đầu vào thứ hai: Request body

```
{
  "action": "TURN_ON"
}
```

Chỉ chấp nhận:
TURN_ON
TURN_OFF

- Luồng xử lý

1. Kiểm tra thiết bị có tồn tại không
2. Tạo action_history với status = PENDING
3. Gửi lệnh qua MQTT
4. ESP32 phản hồi về MQTT
5. Backend cập nhật Database (thành SUCCESS hoặc FAILED)
6. Backend gửi kết quả trạng thái SUCCESS hoặc FAILED về Frontend

- Backend nhận lệnh thành công:

```
HTTP 202 Accepted
{
  "success": true,
  "message": "Lệnh đã được gửi đến thiết bị",
  "data": {
    "actionId": 205,
```

```
    "deviceId": 3,
    "action": "TURN_ON",
    "status": "PENDING",
    "sentAt": "2026-08-18T14:25:00"
  }
}
```

- Không tìm thấy thiết bị:

```
HTTP 404 Not Found
{
  "success": false,
  "error": {
    "code": "DEVICE_NOT_FOUND",
    "message": "Không tìm thấy thiết bị"
  }
}
```

- MQTT không kết nối:

```
HTTP 503 Service Unavailable
{
  "success": false,
  "error": {
    "code": "MQTT_UNAVAILABLE",
    "message": "Không thể kết nối đến MQTT Broker"
  }
}
```

- Đầu ra:

Dữ liệu gửi về Frontend:

```
{
  "actionId": 205,
  "deviceId": 3,
  "action": "TURN_ON",
  "status": "SUCCESS",
  "failureReason": null,
  "responseAt": "2026-08-18T14:25:01"
}
Nếu ESP32 không phản hồi:
{
  "actionId": 205,
  "deviceId": 3,
  "action": "TURN_ON",
  "status": "FAILED",
  "failureReason": "TIMEOUT",
  "responseAt": null
}
```

4.6 API 6: Lấy lịch sử điều khiển

`GET /api/v1/actions`

- Dùng để lấy lịch sử điều khiển 3 thiết bị

- Đầu vào: tham số

Tham số	Bắt buộc	Mặc định	Ý nghĩa
page	Không	1	Trang hiện tại
limit	Không	10	Số bản ghi mỗi trang
search	Không	Rỗng	Tìm theo ID hoặc tên thiết bị

VD:

`GET /api/v1/actions?page=1&limit=10&search=Điều hòa`

- Đầu ra:

```
HTTP 200 OK
```

```
{
 "success": true,
 "data": [
   {
     "id": 205,
     "deviceId": 3,
     "deviceName": "Điều hòa",
     "action": "TURN_ON",
     "status": "SUCCESS",
     "sentAt": "2026-08-18T14:25:00",
     "responseAt": "2026-08-18T14:25:01",
     "failureReason": null
   }
 ],
 "meta": {
   "page": 1,
   "limit": 10,
   "totalItems": 35,
   "totalPages": 4
 }
}
```

- Sai tham số

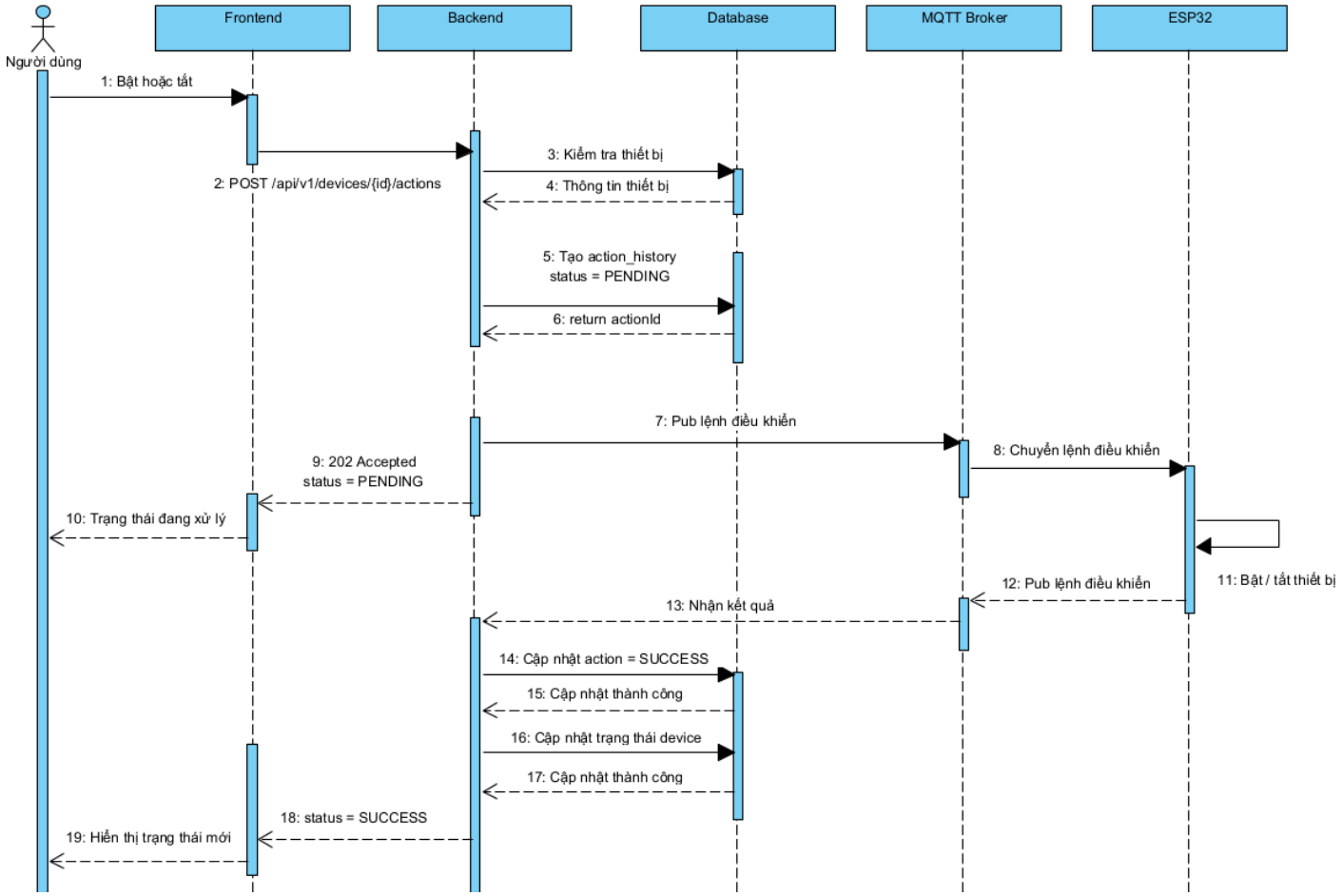
```
HTTP 400 Bad Request
```

- Lỗi Backend

```
HTTP 500 Internal Server Error
```

CHƯƠNG 5. SEQUENCE DIAGRAM

5.1 UC03 - Điều khiển thiết bị

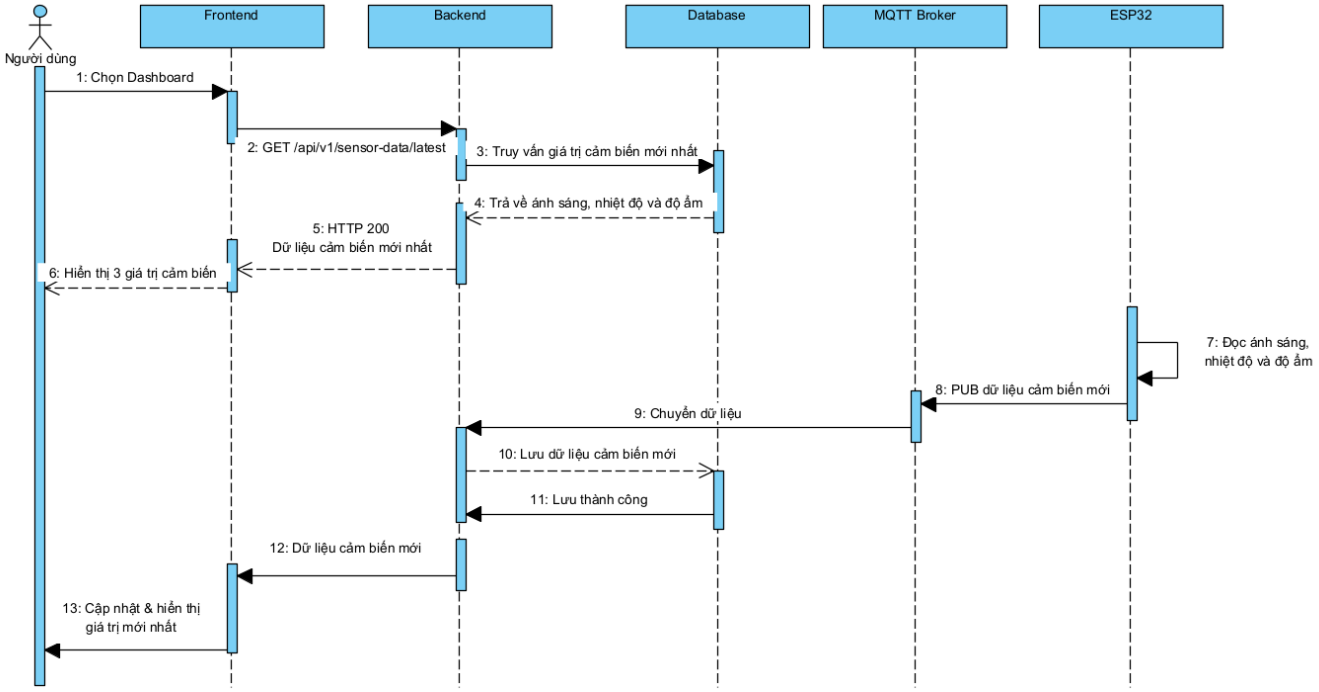


Luồng thực hiện:

1. Người dùng nhấn công tắc bật hoặc tắt một thiết bị trên Dashboard.
2. Frontend gửi yêu cầu điều khiển đến Backend thông qua API POST /api/v1/devices/{id}/actions.
3. Backend kiểm tra thông tin và trạng thái của thiết bị trong Database.
4. Backend tạo bản ghi action_history với trạng thái PENDING.
5. Backend trả về HTTP 202 Accepted để xác nhận đã tiếp nhận lệnh.
6. Frontend hiển thị trạng thái “Đang xử lý”.
7. Backend publish lệnh điều khiển lên MQTT Broker.
8. MQTT Broker chuyển lệnh đến ESP32.
9. ESP32 thực hiện bật hoặc tắt thiết bị.
10. ESP32 publish kết quả thực hiện lên MQTT Broker.
11. MQTT Broker chuyển kết quả về Backend.

12. Backend cập nhật trạng thái lệnh và trạng thái thiết bị trong Database.
13. Backend gửi kết quả SUCCESS cho Frontend thông qua Socket.IO.
14. Frontend cập nhật công tắc thiết bị và hiển thị kết quả cho người dùng.

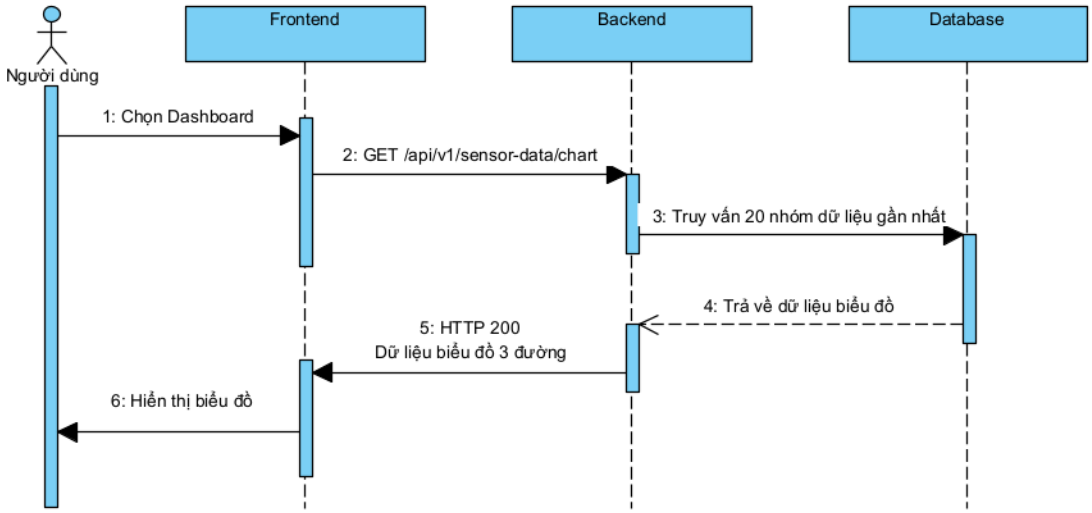
5.2 UC01 – Xem dữ liệu cảm biến thời gian thực



Luồng thực hiện:

1. Người dùng mở màn hình Dashboard.
2. Frontend gửi yêu cầu lấy dữ liệu cảm biến mới nhất đến Backend.
3. Backend truy vấn các giá trị cảm biến gần nhất trong Database.
4. Database trả về dữ liệu ánh sáng, nhiệt độ và độ ẩm.
5. Backend trả dữ liệu cho Frontend thông qua REST API.
6. Frontend hiển thị dữ liệu trên ba thẻ cảm biến.
7. ESP32 định kỳ đọc giá trị từ các cảm biến.
8. ESP32 publish dữ liệu cảm biến lên MQTT Broker.
9. MQTT Broker chuyển dữ liệu đến Backend.
10. Backend lưu dữ liệu cảm biến mới vào Database.
11. Backend gửi dữ liệu mới cho Frontend thông qua Socket.IO.
12. Frontend cập nhật các giá trị cảm biến trên Dashboard.

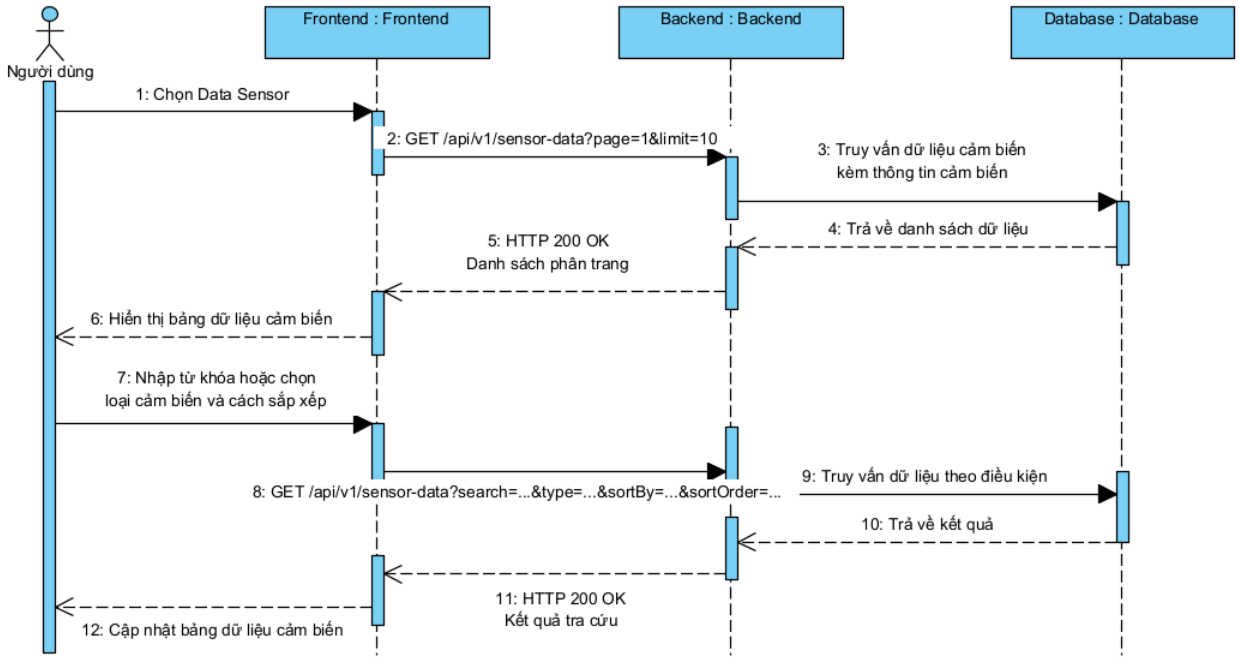
5.3 UC02 – Xem biểu đồ dữ liệu cảm biến



Luồng thực hiện:

1. Người dùng mở màn hình Dashboard.
2. Frontend gửi yêu cầu lấy dữ liệu biểu đồ đến Backend thông qua API GET /api/v1/sensor-data/chart.
3. Backend truy vấn các bản ghi cảm biến gần nhất trong Database.
4. Database trả về dữ liệu ánh sáng, nhiệt độ và độ ẩm theo thứ tự thời gian.
5. Backend xử lý dữ liệu thành ba nhóm tương ứng với ba loại cảm biến.
6. Backend trả dữ liệu biểu đồ cho Frontend.
7. Frontend biểu diễn dữ liệu thành ba đường trên biểu đồ.
8. Người dùng theo dõi sự thay đổi của các giá trị cảm biến theo thời gian.

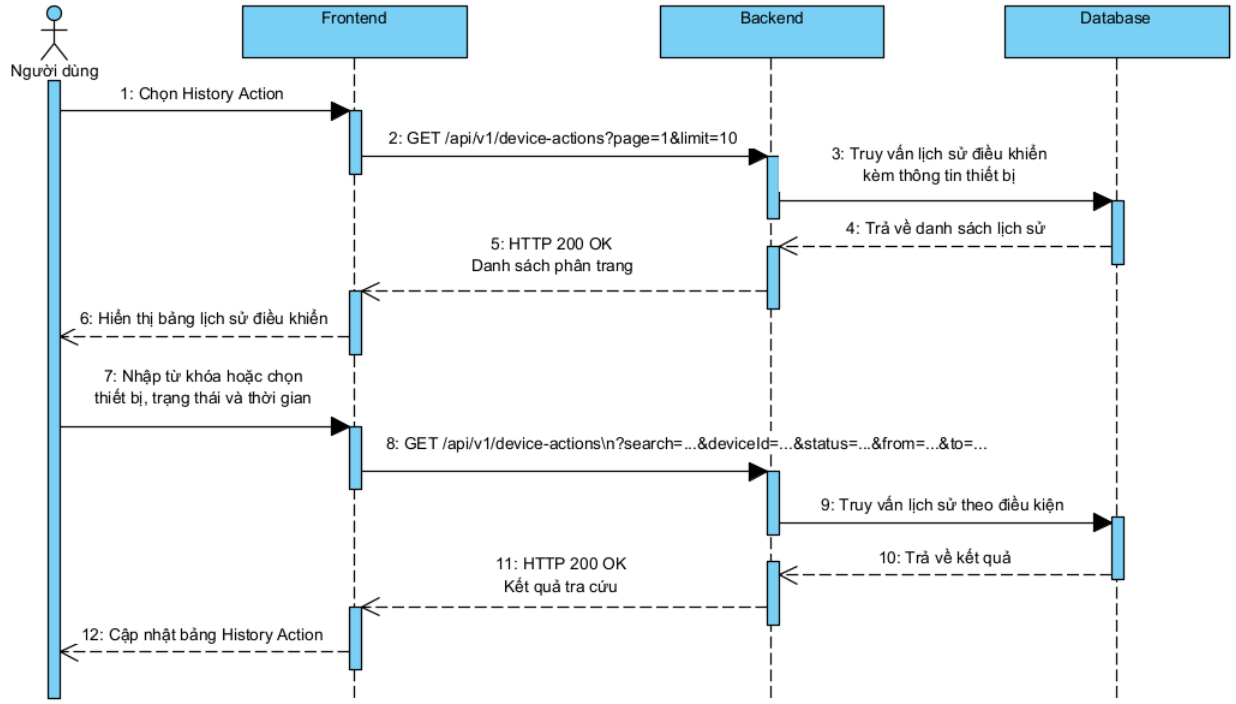
5.4 UC04 -Tra cứu dữ liệu cảm biến



Luồng thực hiện:

1. Người dùng chọn Data Sensor trên thanh menu.
2. Frontend gửi yêu cầu lấy danh sách dữ liệu cảm biến đến Backend.
3. Backend truy vấn dữ liệu cảm biến và thông tin cảm biến trong Database.
4. Database trả về danh sách dữ liệu theo trang.
5. Backend trả kết quả và thông tin phân trang cho Frontend.
6. Frontend hiển thị danh sách trên bảng Data Sensor.
7. Người dùng nhập từ khóa, chọn loại cảm biến hoặc cách sắp xếp.
8. Frontend gửi yêu cầu mới kèm các tham số tìm kiếm và sắp xếp.
9. Backend truy vấn Database theo các điều kiện đã chọn.
10. Database trả về danh sách bản ghi phù hợp.
11. Backend trả kết quả cho Frontend.
12. Frontend cập nhật bảng dữ liệu cảm biến.

5.5 UC05 – Tra cứu lịch sử điều khiển



Luồng thực hiện:

1. Người dùng chọn History Action trên thanh menu.
2. Frontend gửi yêu cầu lấy danh sách lịch sử điều khiển đến Backend.
3. Backend truy vấn bảng action_history và thông tin thiết bị trong Database.
4. Database trả về danh sách lịch sử điều khiển theo trang.
5. Backend trả danh sách và thông tin phân trang cho Frontend.
6. Frontend hiển thị dữ liệu trên bảng History Action.
7. Người dùng nhập ID hoặc tên thiết bị và chọn các điều kiện gồm thiết bị, hành động, trạng thái, thời gian bắt đầu và thời gian kết thúc.
8. Frontend gửi yêu cầu mới kèm các tham số lọc đến Backend.
9. Backend truy vấn Database theo các điều kiện đã chọn.
10. Database trả về các bản ghi lịch sử phù hợp.
11. Backend trả kết quả cho Frontend.
12. Frontend cập nhật bảng History Action.